Esercitazione di laboratorio n. 3

Esercizio n.1: **Elemento maggioritario**

Competenze: ricorsione matematica (Ricorsione e problem-solving: 2.2)

Si scriva una funzione `majority` che, dati un vettore a di N numeri naturali e la sua dimensione, stampi l'elemento maggioritario, se esiste, oppure un messaggio che ne segnali l'assenza. Si assuma N > 0. Si definisce elemento maggioritario, se esiste, quel valore che ha numero di occorrenze > `N`/2.

Esempio: se `N`= 7 e `vet` contiene 3, 3, 9, 4, 3, 5, 3 l'elemento maggioritario è 3. Se `N`=8 e `vet` contiene 0, 1, 0, 2, 3, 4, 0, 5 non esiste elemento maggioritario.
Il prototipo sia:

```
void majority( int *a, int N);
```

Vincoli: si utilizzi un algoritmo ricorsivo *divide et impera*. Non è consentito ordinare il vettore né utilizzare l'algoritmo di Boyer–Moore.

Esercizio n.2: **Anagrafica con liste**

Competenze: creazione e gestione di liste concatenate (Puntatori e strutture dati dinamiche: 4.1)

I dati dell'anagrafica sono memorizzati in un file di testo contenente un numero non specificato di righe. Ogni riga è un record ed ha la forma:

<codice> <nome> <cognome> <data_di_nascita> <via> <citta'> <cap>

La data di nascita è una data valida nella forma gg/mm/aaaa. Nome, cognome, via e città sono stringhe senza spazi di massimo 50 caratteri. Il <cap> è una stringa di cinque cifre, che può iniziare con zero. Il codice è nella forma AXXXX, dove ogni X è una cifra da 0 a 9. Si assuma che i record rispettino questi formati.
I dati di ciascuna persona vanno racchiusi in un opportuno tipo `Item`. L'anagrafica va memorizzata in una lista ordinata per data di nascita crescente: le persone più anziane compaiono prima. A parità di data, si mantenga l'ordine di inserimento. Il codice deve essere univoco nell'anagrafica: se si tenta di inserire un record con un codice già presente, il nuovo record non viene inserito.

Si scriva un programma in C che inizializzi una lista vuota e offra le seguenti funzionalità:

1. acquisizione da tastiera e inserimento ordinato di un nuovo record;
2. acquisizione da file e inserimento ordinato dei nuovi record: si esaminano tutte le righe del file. Se il codice di un record letto dal file è già presente nell'anagrafica, il record viene scartato. Se più record del file hanno lo stesso codice, viene inserito solo il primo, sempre se quel codice non è già presente nell'anagrafica; i successivi vengono scartati. La presenza di duplicati non interrompe la lettura;
3. ricerca di un record per codice;
4. cancellazione, con estrazione del dato, di un record individuato per codice;
5. cancellazione, con estrazione dei dati, di tutti i record con data di nascita compresa tra due date, estremi inclusi, lette da tastiera;
6. stampa della lista su file nel formato dei record sopra indicato, in modo che il file possa essere successivamente riletto dal programma.

Si assuma che, per l'operazione 5, la prima data immessa sia minore o uguale alla seconda. La ricerca e le funzioni di cancellazione non stampano i dati: rendono disponibile al programma chiamante il record trovato o estratto, oppure segnalano che non esiste un record corrispondente. La cancellazione rimuove il nodo dalla lista; i dati dell'`Item` estratto devono restare disponibili al chiamante anche dopo la rimozione.

Suggerimento: per l'operazione 5 si può realizzare una funzione che, a ogni chiamata, estragga il primo record della lista la cui data rientra nell'intervallo, oppure segnali che non ne rimangono altri. Il programma richiama la funzione e stampa i record estratti fino a tale segnalazione.